

Module 9: Strings

9.1 C-Style Strings

- A C-style string is an array of characters ending with a special null character written as `'\0'`
- The null character marks the end of the string
- Declaration examples
`char name[20]; // can hold up to 19 characters plus the null terminator`
`char city[] = "Karachi"; // automatically adds '\0' at the end`
- Reading with `cin`: reads one word (stops at space)
- Reading with `cin.getline(charArray, size)`: reads a full line up to size-1 characters
- Common functions from the `cstring` header
 - `strlen(str)`: returns the number of characters before the null terminator
 - `strcpy(dest, src)`: copies `src` into `dest`; `dest` must be large enough
 - `strcat(dest, src)`: appends `src` to the end of `dest`
 - `strcmp(s1, s2)`: compares two strings; returns 0 if equal, negative if `s1 < s2`, positive if `s1 > s2`
 - `strchr(str, ch)`: finds first occurrence of `ch` in `str`
 - `strstr(str, sub)`: finds first occurrence of `sub` in `str`

9.2 The C++ string Class

- The `string` class from the `string` header provides a safer and more convenient way to work with text
- A `string` object manages its own memory and can grow or shrink automatically
- Declaration
`string name;`
`string city = "Lahore";`
- Reading
`cin >> word; // reads one word`
`getline(cin, fullName); // reads entire line`
- Concatenation using the `+` operator
`string full = firstName + " " + lastName;`
- Comparing strings using `==` and `!=` operators (unlike C-style strings, this works correctly)
- Accessing individual characters using square bracket notation: `name[0]`, `name[1]`
- Useful `string` class methods
 - `length()` or `size()`: returns the number of characters
 - `empty()`: returns true if the string has no characters
 - `substr(start, length)`: extracts a portion of the string

- `find(substring)`: returns the position of the first match or `string::npos` if not found
- `replace(start, length, newStr)`: replaces characters in the string
- `erase(start, length)`: removes characters
- `append(str)`: adds `str` to the end
- `at(index)`: accesses character with bounds checking (throws exception if out of range)
- `c_str()`: returns a pointer to a null-terminated C-style version of the string

9.3 String Practice Examples

- Count the number of vowels in a sentence
- Count the number of words in a sentence (count spaces plus one)
- Reverse a string
- Check if a string is a palindrome (reads the same forwards and backwards)
- Convert all characters to uppercase or lowercase using `toupper()` or `tolower()` from `cctype`
- Remove all spaces from a string